



INSA SUMMER CAMP

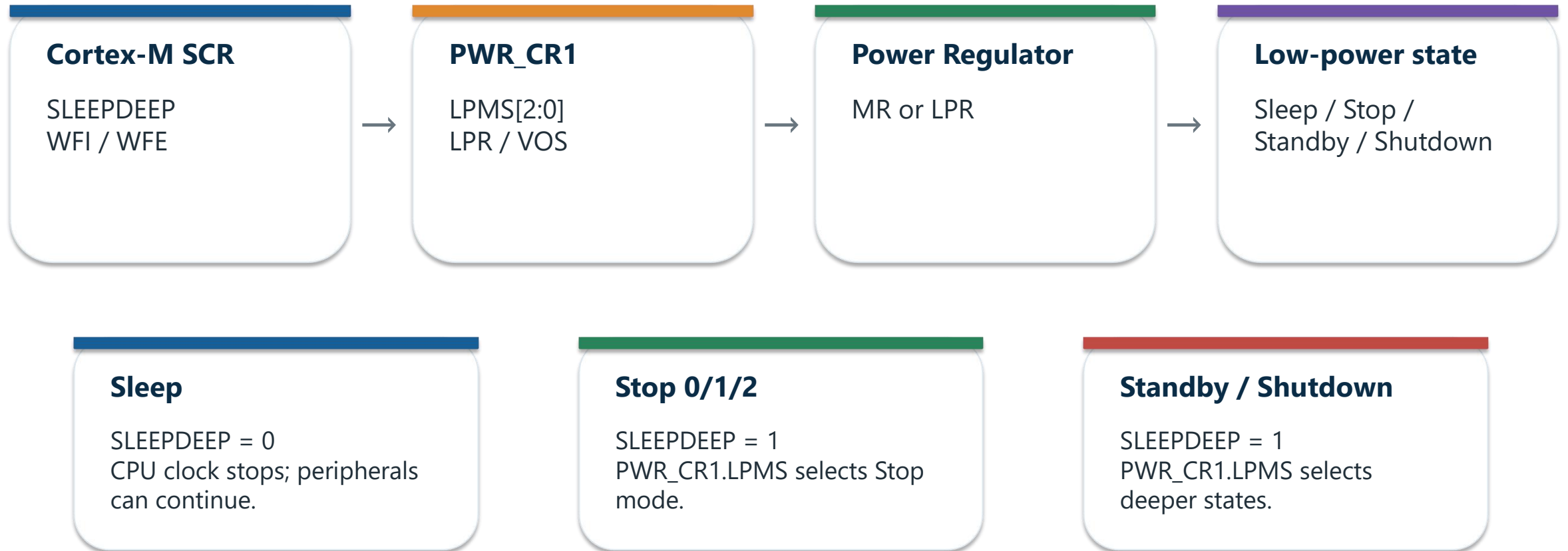
STM32L476RG

STM32 Low-Power Modes and Wake-Up Mechanisms

Name: Haimanot Desie
ID: CTC-2130-2

Power-Control Architecture

STM32L4 separates CPU sleep control from deeper power-domain selection



Low-Power Mode Comparison

Requested STM32L476RG modes

Mode	CPU	Clock / regulator behavior	Retention / wake-up focus
Sleep	Stopped	Normal clocks available to peripherals	Fast interrupt/event wake; state retained
Low-power Sleep	Stopped	Low-power run regulator configuration	Wake returns to Low-power Run
Stop 0	Stopped	VCORE clocks stopped; main regulator remains on	Fast wake; SRAM/registers retained
Stop 1	Stopped	Deeper regulation than Stop 0	More power saving; many wake sources
Stop 2	Stopped	VCORE in lower-leakage state	Very low power; SRAM/registers retained
Standby	Off	VCORE powered off	Main core state lost; RTC/wakeup domain can remain
Shutdown	Off	Regulator/VCORE power removed further	Deepest power saving; limited wake pins

Source basis: STM32L476RG DS10198 and RM0351.

RCC Registers and Clock Control

Clock gating is central to reducing dynamic power

RCC_CR

Clock source control

- HSE
- HSI16
- MSI
- PLL

Low-power modes disable or preserve clock sources according to the selected state.

RCC_CFGR / PLLCFGR

System clock selection and PLL configuration.

When returning from Stop, the system clock source must be re-established as required by the software design.

Peripheral clock enables

AHB/APB enable registers control whether peripheral clocks are supplied.

Examples:

- RCC_AHB2ENR — GPIO clocks
- RCC_APB1ENR1/2 — PWR, timers, communication blocks
- RCC_APB2ENR — higher-speed peripheral clocks

PWR Registers and Important Bits

Bit-level view of how the STM32L476RG selects low-power states

PWR_CR1

LPMS[2:0] — selects Stop 0, Stop 1, Stop 2, Standby or Shutdown

LPR — selects low-power regulator mode

VOS[1:0] — voltage-scaling selection

DBP — allows backup-domain write access

PWR_CR3

RRS — SRAM2 retention in Standby configuration

EWUPx — wake-up pin enable bits

EIWUL — internal wake-up line enable

APC — apply pull-up/pull-down configuration

PWR_SCR / Status

PWR_SCR: clear wake-up flags

PWR_SR1 / PWR_SR2: wake/low-power status information

Engineering sequence:

1. Configure wake source
2. Select PWR mode
3. Set SLEEPDEEP
4. Execute WFI/WFE

Cortex-M4 SCR: Sleep vs Deep Sleep

The SCR decides whether WFI/WFE enters ordinary Sleep or the STM32 deep-sleep path

SLEEPDEEP (bit 2)

0 → Sleep

1 → Deep Sleep

STM32 Stop, Standby and Shutdown are selected through the deep-sleep path and PWR configuration.

SEVONPEND (bit 4)

Controls whether pending interrupts can generate an event that wakes the processor from WFE.

Useful when designing event-driven low-power software.

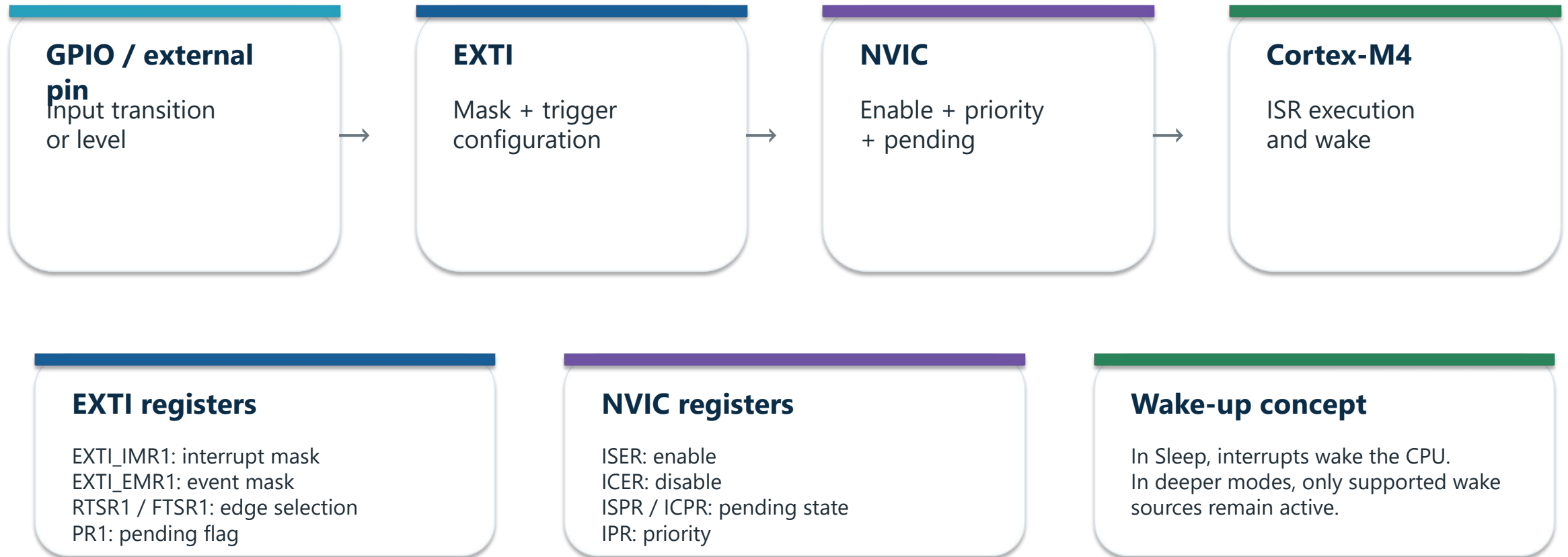
SLEEPONEXIT (bit 1)

1 → after an ISR, the core can return directly to sleep/deep sleep.

Useful for interrupt-driven systems that do little work in Thread mode.

EXTI and NVIC: Interrupt Wake-Up Path

External pins become wake-up requests through EXTI and the Cortex-M NVIC



Wake-Up Sources on STM32L476RG

External interrupts / EXTI

GPIO edges, interrupt lines

Wake-up pins

Dedicated WKUP pins for deep modes

RTC

Alarm, periodic wake-up and related events

Low-power timers

LPTIM can remain available for selected Stop use cases

Independent peripherals

Examples include LPUART / selected I2C wake-up capability

Watchdogs

IWDG and reset-related wake behavior where supported

Selection rule: always check the RM0351 wake-up table for the exact low-power mode and peripheral combination.

WFI (Wait For Interrupt)

Interrupt-driven low-power entry

How WFI works

1. CPU completes current instructions.
2. Software executes `__WFI()`.
3. Core enters Sleep or Deep Sleep according to SCR.
4. A qualifying interrupt/exception wakes the processor.
5. ISR executes, then software resumes.

Typical use

Sensor node waits for an interrupt

Button / EXTI → wake → ISR → process → sleep again

Key advantage

Simple mental model: "sleep until an interrupt needs attention."

Best suited to interrupt-driven applications where the wake-up condition is explicit.

WFE (Wait For Event)

Event-driven low-power entry

How WFE works

1. CPU executes __WFE().
2. If an event is already pending, execution can continue immediately.
3. Otherwise the core waits for an event.
4. Events can come from event-enabled sources or SEV-related mechanisms.

Typical use

Synchronization between execution contexts or hardware events

Useful when an application cares about “an event happened,” not only a serviceable interrupt.

Key distinction from WFI

WFI is centered on interrupt wake-up.

WFE uses the Cortex-M event mechanism and the event register; EXTI event mode and SEV/SEVONPEND can participate.

WFI vs WFE Comparison

Aspect	WFI	WFE
Primary concept	Wait for interrupt	Wait for event
Common wake source	Enabled interrupt/exception	Event signal / event register
Core register	SCR controls Sleep/Deep Sleep	SCR + event mechanism
Good fit	Interrupt-driven firmware	Event-driven synchronization
Deep sleep support	Yes, with SLEEPDEEP=1	Yes, with SLEEPDEEP=1

Practical Applications

Matching low-power behavior to real embedded-system requirements

Battery sensor node

Sleep most of the time; wake on GPIO or RTC; sample and transmit.

Industrial monitor

Use timers/RTC and external events to wake only when work is needed.

Remote control / security node

Dedicated wake-up pin or GPIO event triggers immediate processing.

Wearable / portable device

Stop modes reduce average current while preserving SRAM and fast wake.

Smart meter / data logger

RTC schedules measurements; MCU returns to deep low-power state.

Asset tracker

Stop/Standby with RTC scheduling minimizes battery drain over long periods.

Comparison (ATmega328P vs STM32L476RG)

Feature	ATmega328P	STM32L476RG
Sleep modes	Idle, ADC Noise Reduction, Power-down, Power-save, Standby / Extended Standby	Sleep, Low-power Sleep, Stop 0/1/2, Standby, Shutdown
Deepest low-power mode	Power-down	Shutdown
Sleep-control registers	SMCR; MCUCR where applicable	PWR registers + RCC + Cortex-M SCR
Wake-up sources	INT0/INT1, pin-change, watchdog; Timer2/async sources in relevant modes	EXTI/GPIO, wake-up pins, RTC, LPTIM and selected peripherals, watchdog/reset sources
External interrupt wake-up	Yes; INT0 can wake Power-down with appropriate sensing configuration	Yes; EXTI/NVIC and dedicated wake mechanisms
Timer operation during sleep	Depends on mode; Timer2 can run asynchronously in Power-save	Depends on mode; low-power timers and selected clocks can remain active
RTC operation	Timer2 can emulate RTC with external 32.768 kHz crystal; no dedicated RTC peripheral	Dedicated RTC with calendar, alarms and backup domain
RAM retention	Retained through AVR sleep modes; reset/deeper power removal is different	SRAM retained in Stop; core state is lost in Standby/Shutdown, with limited retention options
Typical applications	Simple battery nodes, AVR control, low-cost sensors	Battery IoT, wearables, data loggers, industrial and connected systems

Main Objective

